Assignment 3:
Improvement methods of my failure results for Lid-Driven Cavity

Chen Peng Chung

2025.12.05

Content

1. Problem Description
2. TDMA algorithm
3. Alternating-Direction Implicit (ADI) Technique
4. Results and Discussion
5. Conclusion

1. Problem descriptions

1.**Oscillation** :

A significant problem with my TVD-MUSCL scheme is the oscillatory nature of the centerline velocity profiles.

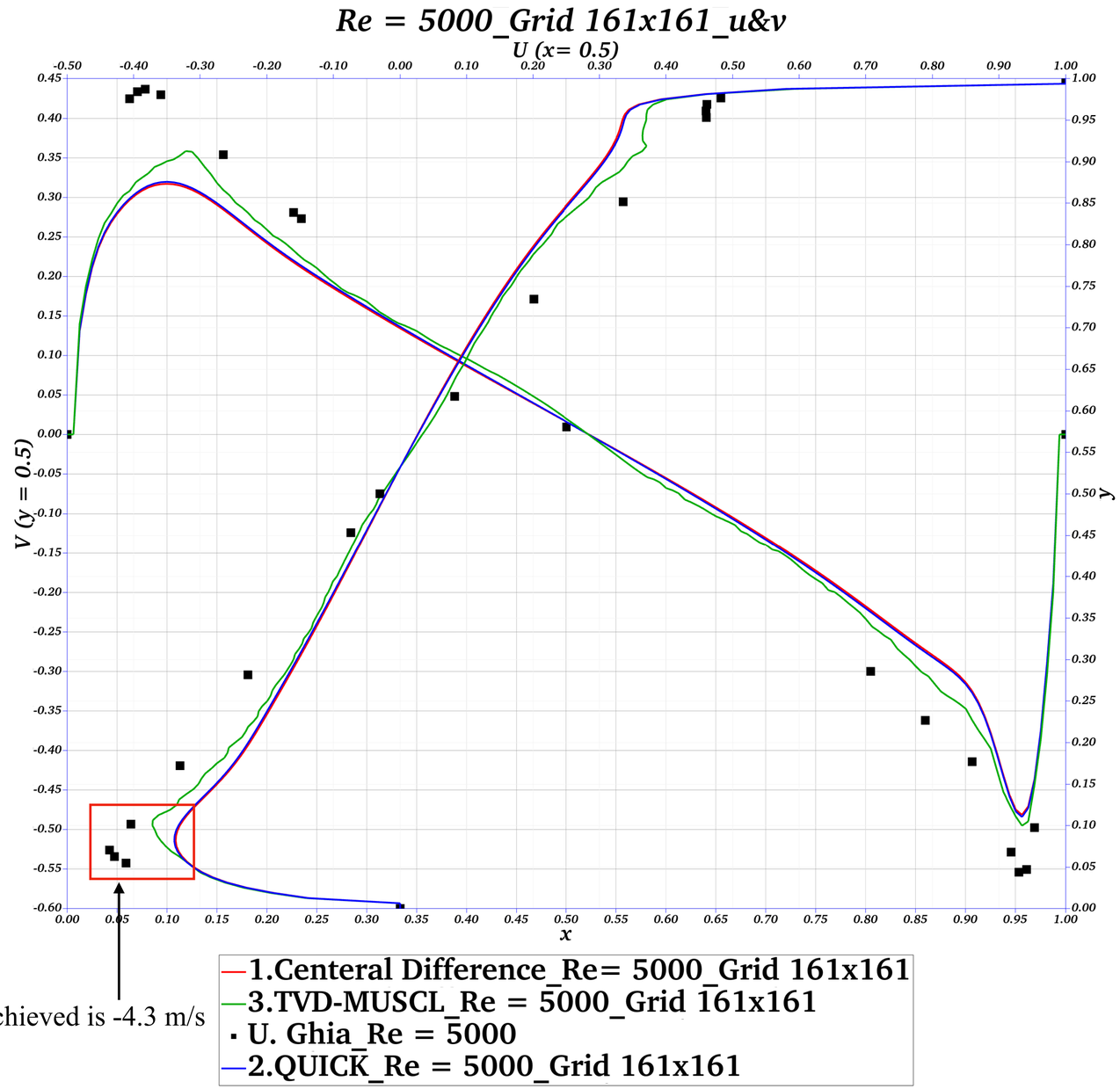
2.**BottomLeft x-Velocity** :

All of my x-velocity results fail to achieve the target value of -4.3 at the bottom-left corner shown in the right figure. This is the primary difficulty I aim to address.

3.Computational Time Used :

Originally, my first code took at least 3 days to obtain a converged solution, which was very inefficient. The limitations of computational resources are issues we must consider.

Method changed the dilemma : ADI method with TDMA algorithm



The minimum x-velocity achieved is -4.3 m/s

2. TDMA algorithm : 2 step in programming

1. **The TDMA algorithm is used to solve the tridiagonal matrix.**
2. **This is a directly solution method.**
3. **This method is the key to completing Assignment 3 within the time limit, which is why our teacher emphasized it repeatedly.**

1. Easy math : step 1 - forward elimination

(1) Our known : Boundary Condition , that are the point 0 and point 7 values

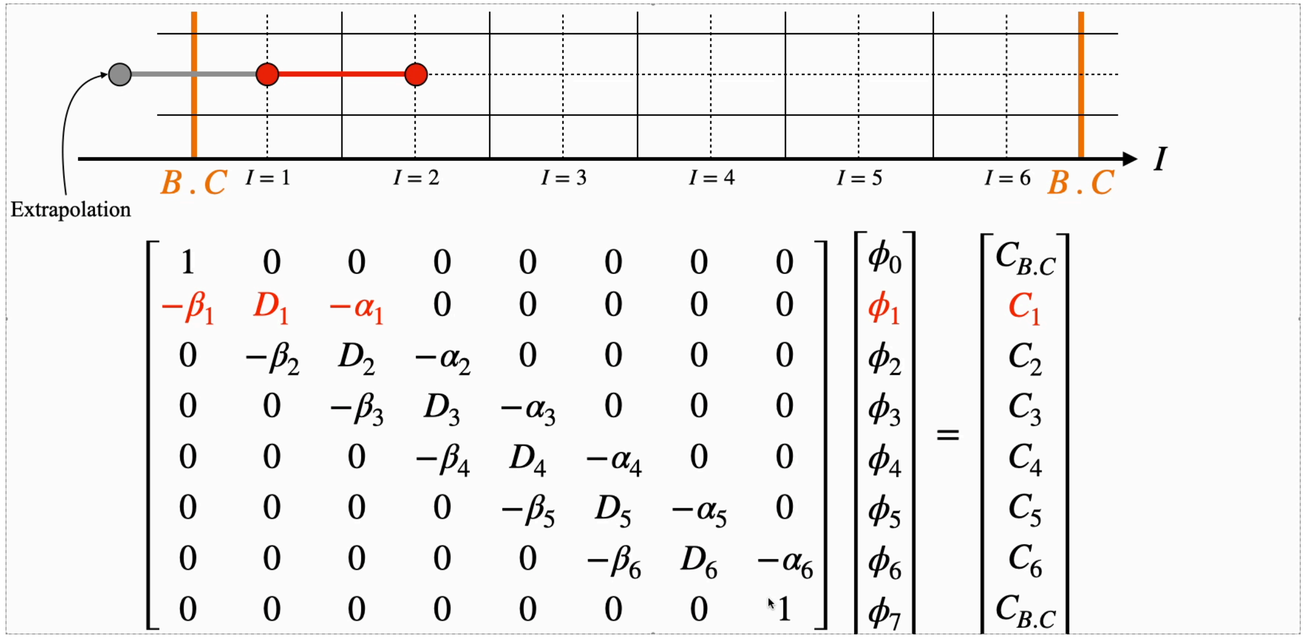
(2) Unknown Range : point 1 ~ point 6 at the right figure.

$$\phi_1 = \frac{\alpha_1}{D_1}\phi_2 + \frac{\beta_1}{D_1}\phi_{B.C} + \frac{C_1}{D_1} \qquad (2.1)$$

$$\phi_2 = \frac{\alpha_2}{D_2}\phi_3 + \frac{\beta_2}{D_2}\phi_1 + \frac{C_2}{D_2} \qquad (2.2)$$

$$\phi_3 = \frac{\alpha_3}{D_3}\phi_4 + \frac{\beta_3}{D_3}\phi_2 + \frac{C_3}{D_3} \qquad (2.3)$$

$$\phi_n = \frac{\alpha_n}{D_n}\phi_{n+1} + \frac{\beta_n}{D_n}\phi_{n-1} + \frac{C_n}{D_n} \qquad (2.4)$$



$$\begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ -\beta_1 & D_1 & -\alpha_1 & 0 & 0 & 0 & 0 & 0 \\ 0 & -\beta_2 & D_2 & -\alpha_2 & 0 & 0 & 0 & 0 \\ 0 & 0 & -\beta_3 & D_3 & -\alpha_3 & 0 & 0 & 0 \\ 0 & 0 & 0 & -\beta_4 & D_4 & -\alpha_4 & 0 & 0 \\ 0 & 0 & 0 & 0 & -\beta_5 & D_5 & -\alpha_5 & 0 \\ 0 & 0 & 0 & 0 & 0 & -\beta_6 & D_6 & -\alpha_6 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \phi_0 \\ \phi_1 \\ \phi_2 \\ \phi_3 \\ \phi_4 \\ \phi_5 \\ \phi_6 \\ \phi_7 \end{bmatrix} = \begin{bmatrix} C_{B.C} \\ C_1 \\ C_2 \\ C_3 \\ C_4 \\ C_5 \\ C_6 \\ C_{B.C} \end{bmatrix}$$

▲ Figure 2.1 TDMA in 1-D

2. TDMA algorithm

So you will get the general form of the elimination :

$$\phi_j = A_j \phi_{j+1} + C'_j \qquad (2.5)$$

Where,

$$A_j = \frac{\alpha_j}{D_j - \beta_j A_{j-1}} \ , \ C'_j = \frac{\beta_j C'_{j-1} + C_j}{D_j - \beta_j A_{j-1}}$$

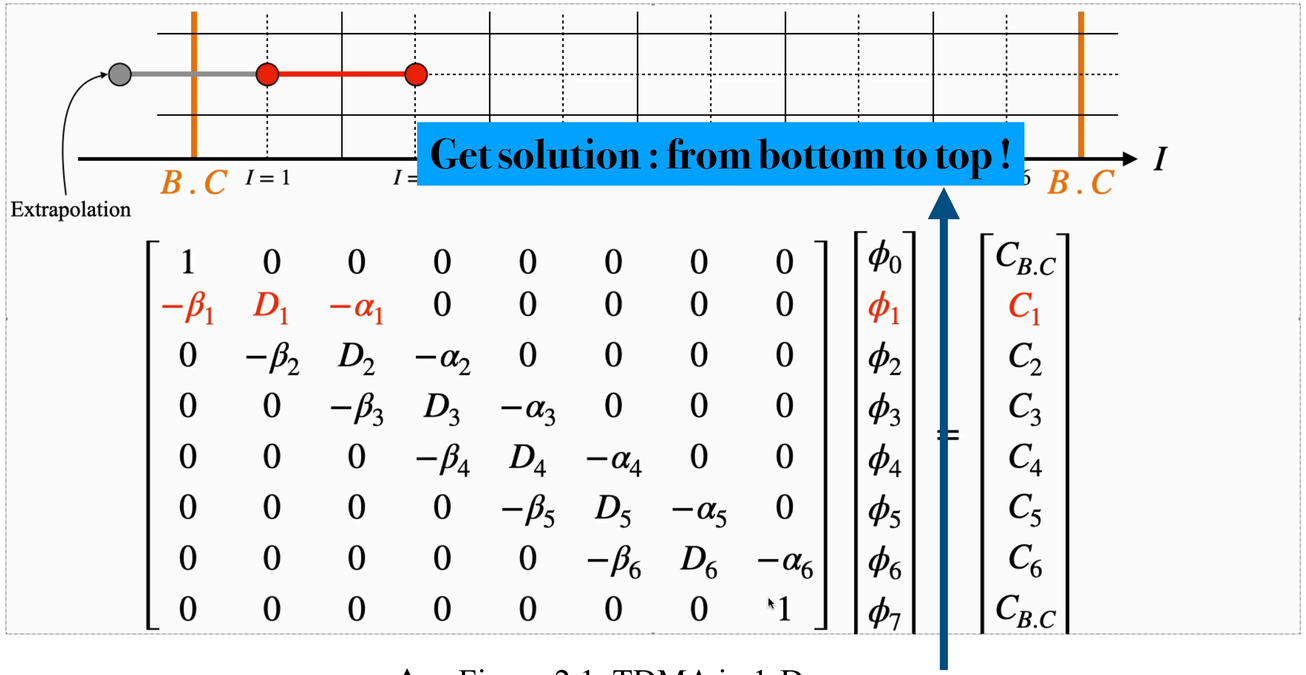
2. Easy math : step 2- back-substitution

At the last equation, you must use the boundary condition :

$$\phi_6 = A_6 \phi_7 + C'_6 \qquad (2.6)$$

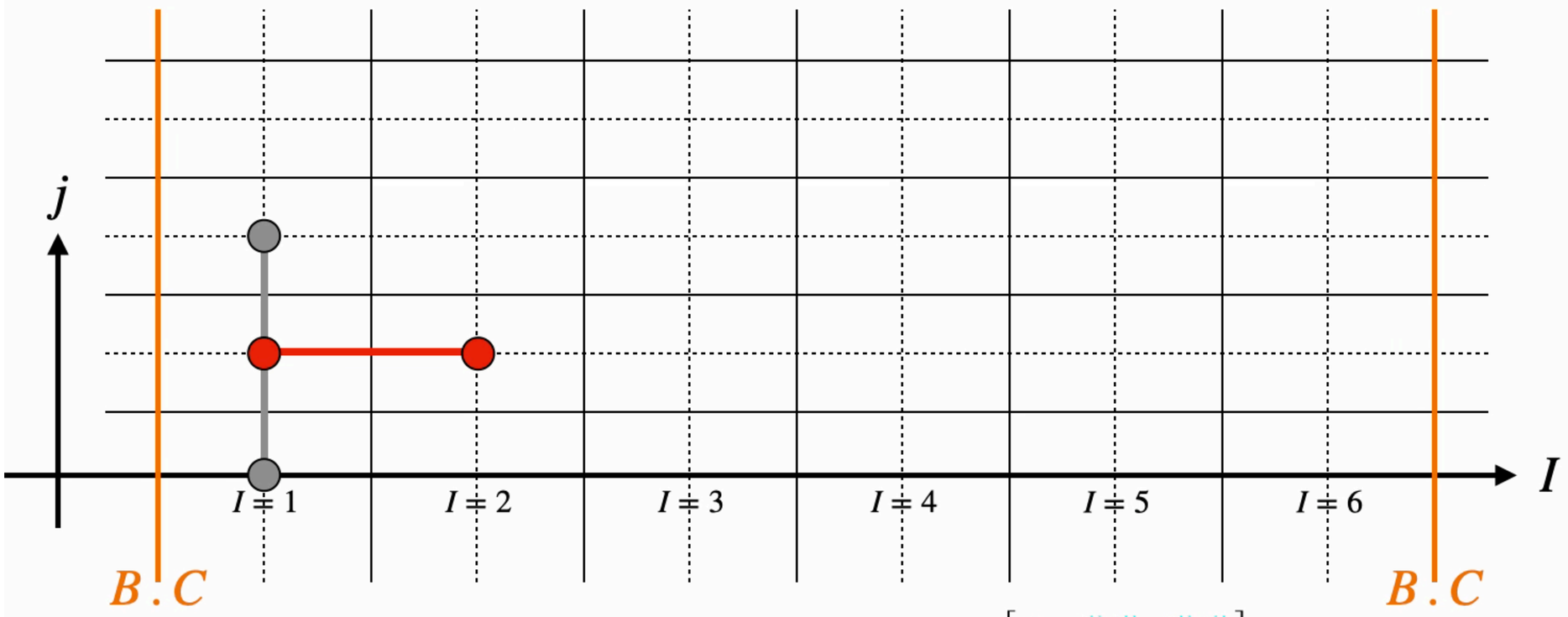
$$[\ \phi_6 \rightarrow \phi_5 \rightarrow \phi_4 \rightarrow \phi_3 \rightarrow \phi_2 \rightarrow \phi_1]$$

Finally, for the bottom equation, using right hand side B.C, and getting all solution of the system are called as “back-substitution”.



▲ Figure 2.1 TDMA in 1-D

2. TDMA algorithm ：appling the method in 2-D matrix system



$$
\begin{bmatrix}
D_1 & -\alpha_1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
-\beta_2 & D_2 & -\alpha_2 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & -\beta_3 & D_3 & -\alpha_3 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & -\beta_4 & D_4 & -\alpha_4 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & -\beta_5 & D_5 & -\alpha_5 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & -\beta_6 & D_6 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & D_7 & -\alpha_7 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & -\beta_8 & D_8 & -\alpha_8 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & -\beta_9 & D_9 & -\alpha_9 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & -\beta_{10} & D_{10} & -\alpha_{10} & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & -\beta_{11} & D_{11} & -\alpha_{11} \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & -\beta_{12} & D_{12}
\end{bmatrix}
\begin{bmatrix}
\phi_1 \\ \phi_2 \\ \phi_3 \\ \phi_4 \\ \phi_5 \\ \phi_6 \\ \phi_7 \\ \phi_8 \\ \phi_9 \\ \phi_{10} \\ \phi_{11} \\ \phi_{12}
\end{bmatrix}
=
\begin{bmatrix}
C_1 + \gamma_1^{old}\phi_{N,1}^{old} + \delta_1^{old}\phi_{S,1}^{old} \\
C_2 + \gamma_2^{old}\phi_{N,2}^{old} + \delta_2^{old}\phi_{S,2}^{old} \\
C_3 + \gamma_3^{old}\phi_{N,3}^{old} + \delta_3^{old}\phi_{S,3}^{old} \\
C_4 + \gamma_4^{old}\phi_{N,4}^{old} + \delta_4^{old}\phi_{S,4}^{old} \\
C_5 + \gamma_5^{old}\phi_{N,5}^{old} + \delta_5^{old}\phi_{S,5}^{old} \\
C_6 + \gamma_6^{old}\phi_{N,6}^{old} + \delta_6^{old}\phi_{S,6}^{old} \\
C_7 + \gamma_7^{old}\phi_{N,7}^{old} + \delta_7^{old}\phi_{S,7}^{old} \\
C_8 + \gamma_8^{old}\phi_{N,8}^{old} + \delta_8^{old}\phi_{S,8}^{old} \\
C_9 + \gamma_9^{old}\phi_{N,9}^{old} + \delta_9^{old}\phi_{S,9}^{old} \\
C_{10} + \gamma_{10}^{old}\phi_{N,10}^{old} + \delta_{10}^{old}\phi_{S,10}^{old} \\
C_{11} + \gamma_{11}^{old}\phi_{N,11}^{old} + \delta_{11}^{old}\phi_{S,11}^{old} \\
C_{12} + \gamma_{12}^{old}\phi_{N,12}^{old} + \delta_{12}^{old}\phi_{S,12}^{old}
\end{bmatrix}
$$

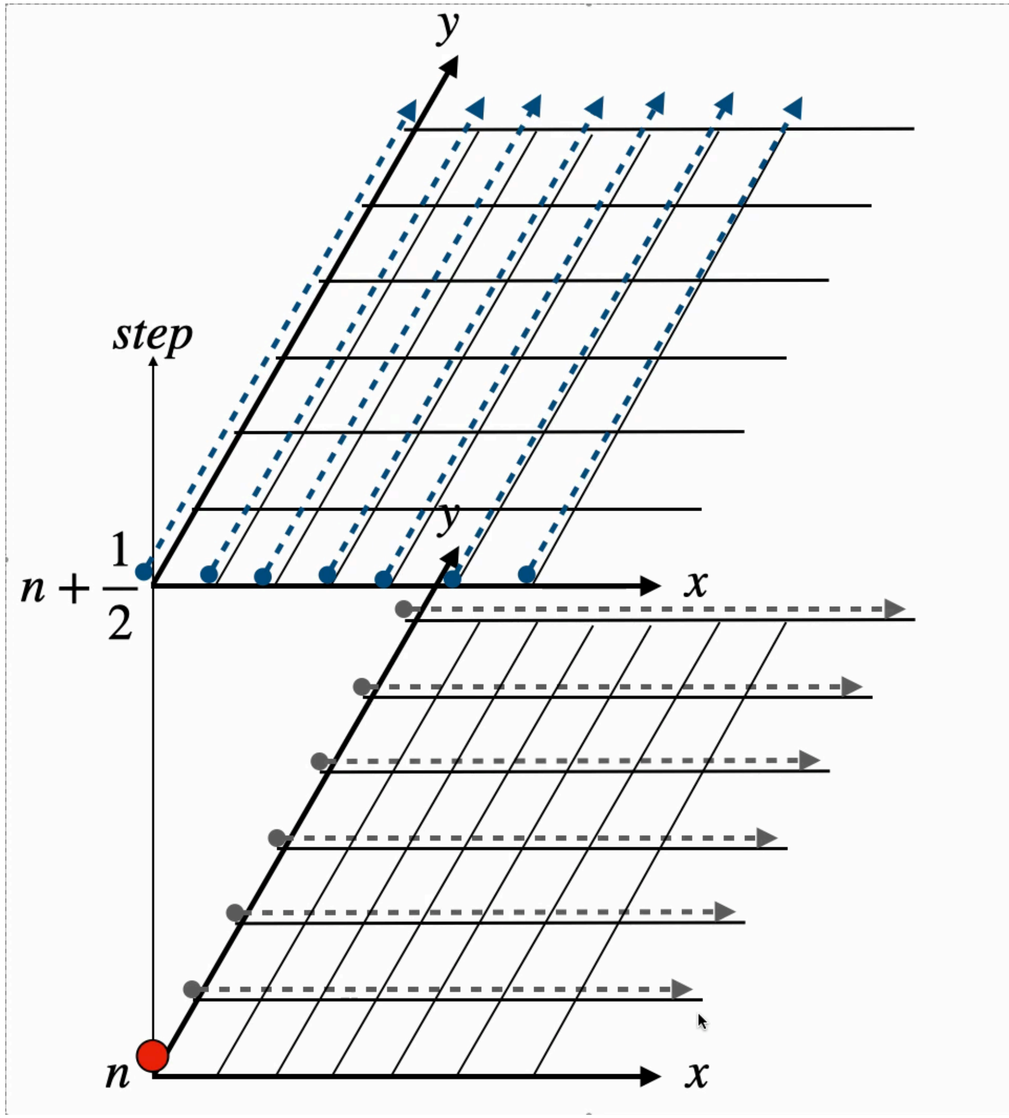
3. Alternating-Direction Implicit (ADI) Technique

In code implementation, the ADI method can be divided into two steps:

Step 1: Choose one direction (e.g., x-direction) and move the neighbor values from the other direction (y-direction) to the source term

Step 2: Solve the resulting tridiagonal matrix system along the chosen direction. Then switch directions and repeat to obtain the n+1 solution from the intermediate n+1/2 solution.

SUMMARY : ADI method, from n to n+0.5, get solution from left to right in inner scanning direction; from n+0.5 to n + 1, get solution from bottom to top in inner scanning direction;



3. Alternating-Direction Implicit (ADI) Technique

For example : pressure correction in SIMPLE

• Split the equation into two parts and scan the matrix in different directions.

$$\left(\frac{\delta y}{\rho(a_P)^*_{i+1,J}}+\frac{\delta y}{\rho(a_P)^*_{i,J}}+\frac{\delta x}{\rho(b_P)^*_{I,j+1}}+\frac{\delta x}{\rho(b_P)^*_{I,j}}\right)p'^{N+1}_{I,J}-\left(\left(\frac{\delta x}{\rho(b_P)^*_{I,j}}\right)p'^{N}_{I,J-1}+\left(\frac{\delta x}{\rho(b_P)^*_{I,j+1}}\right)p'^{N}_{I,J+1}+\left(\frac{\delta y}{\rho(a_P)^*_{i,J}}\right)p'^{N}_{I-1,J}+\left(\frac{\delta y}{\rho(a_P)^*_{i+1,J}}\right)p'^{N}_{I+1,J}\right)=S$$

Scanning direction 1 : from left to right :

$$\left(\frac{\delta y}{\rho(a_P)^*_{i+1,J}}+\frac{\delta y}{\rho(a_P)^*_{i,J}}+\right)p'^{N+\frac{1}{2}}_{I,J}-\left(\left(\frac{\delta y}{\rho(a_P)^*_{i,J}}\right)p'^{N+\frac{1}{2}}_{I-1,J}+\left(\frac{\delta y}{\rho(a_P)^*_{i+1,J}}\right)p'^{N+\frac{1}{2}}_{I+1,J}\right)=S\left(\frac{\delta x}{\rho(b_P)^*_{I,j+1}}+\frac{\delta x}{\rho(b_P)^*_{I,j}}\right)p'^{N}_{I,J}-\left(\left(\frac{\delta x}{\rho(b_P)^*_{I,j}}\right)p'^{N}_{I,J-1}+\left(\frac{\delta x}{\rho(b_P)^*_{I,j+1}}\right)p'^{N}_{I,J+1}+)p'^{N}_{I+1,J}\right)$$

Scanning direction 2 : from bottom to top :

$$\left(\frac{\delta x}{\rho(b_P)^*_{I,j+1}}+\frac{\delta x}{\rho(b_P)^*_{I,j}}\right)p'^{N+1}_{I,J}-\left(\left(\frac{\delta x}{\rho(b_P)^*_{I,j}}\right)p'^{N+1}_{I,J-1}+\left(\frac{\delta x}{\rho(b_P)^*_{I,j+1}}\right)p'^{N+1}_{I,J+1}\right)=S\left(\frac{\delta y}{\rho(a_P)^*_{i+1,J}}+\frac{\delta y}{\rho(a_P)^*_{i,J}}\right)p'^{N+\frac{1}{2}}_{I,J}-\left(\left(\frac{\delta y}{\rho(a_P)^*_{i,J}}\right)p'^{N+\frac{1}{2}}_{I-1,J}+\left(\frac{\delta y}{\rho(a_P)^*_{i+1,J}}\right)p'^{N+\frac{1}{2}}_{I+1,J}\right)$$

4. Results and Discussion

Two things I wanted to present :

1. The centerline velocity distribution of u and v for Grid 81x81, 113x113, 161x161, 227x227 at Re = 5000.
2. Accuracy comparison for CDs, QUICK, TVD-MUSCL :

$$L_1 norm = \frac{1}{34} \sum_{i=1}^{17} \sum_{j=1}^{17} \left(|u - u_{Ghia}| + |v - v_{Ghia}| \right)$$

129-grid pt. no.	y	Re = 100	400	1000	3200	5000	7500	10,000
129	1.00000	1.00000	1.00000	1.00000	1.00000	1.00000	1.00000	1.00000
126	0.9766	0.84123	0.75837	0.65928	0.53236	0.48223	0.47244	0.47221
125	0.9688	0.78871	0.68439	0.57492	0.48296	0.46120	0.47048	0.47783
124	0.9609	0.73722	0.61756	0.51117	0.46547	0.45992	0.47323	0.48070
123	0.9531	0.68717	0.55892	0.46604	0.46101	0.46036	0.47167	0.47804
110	0.8516	0.23151	0.29093	0.33304	0.34682	0.33556	0.34228	0.34635
95	0.7344	0.00332	0.16256	0.18719	0.19791	0.20087	0.20591	0.20673
80	0.6172	−0.13641	0.02135	0.05702	0.07156	0.08183	0.08342	0.08344
65	0.5000	−0.20581	−0.11477	−0.06080	−0.04272	−0.03039	−0.03800	0.03111
59	0.4531	−0.21090	−0.17119	−0.10648	−0.86636	−0.07404	−0.07503	−0.07540
37	0.2813	−0.15662	−0.32726	−0.27805	−0.24427	−0.22855	−0.23176	−0.23186
23	0.1719	−0.10150	−0.24299	−0.38289	−0.34323	−0.33050	−0.32393	−0.32709
14	0.1016	−0.06434	−0.14612	−0.29730	−0.41933	−0.40435	−0.38324	−0.38000
10	0.0703	−0.04775	−0.10338	−0.22220	−0.37827	−0.43643	−0.43025	−0.41657
9	0.0625	−0.04192	−0.09266	−0.20196	−0.35344	−0.42901	−0.43590	−0.42537
8	0.0547	−0.03717	−0.08186	−0.18109	−0.32407	−0.41165	−0.43154	−0.42735
1	0.0000	0.00000	0.00000	0.00000	0.00000	0.00000	0.00000	0.00000

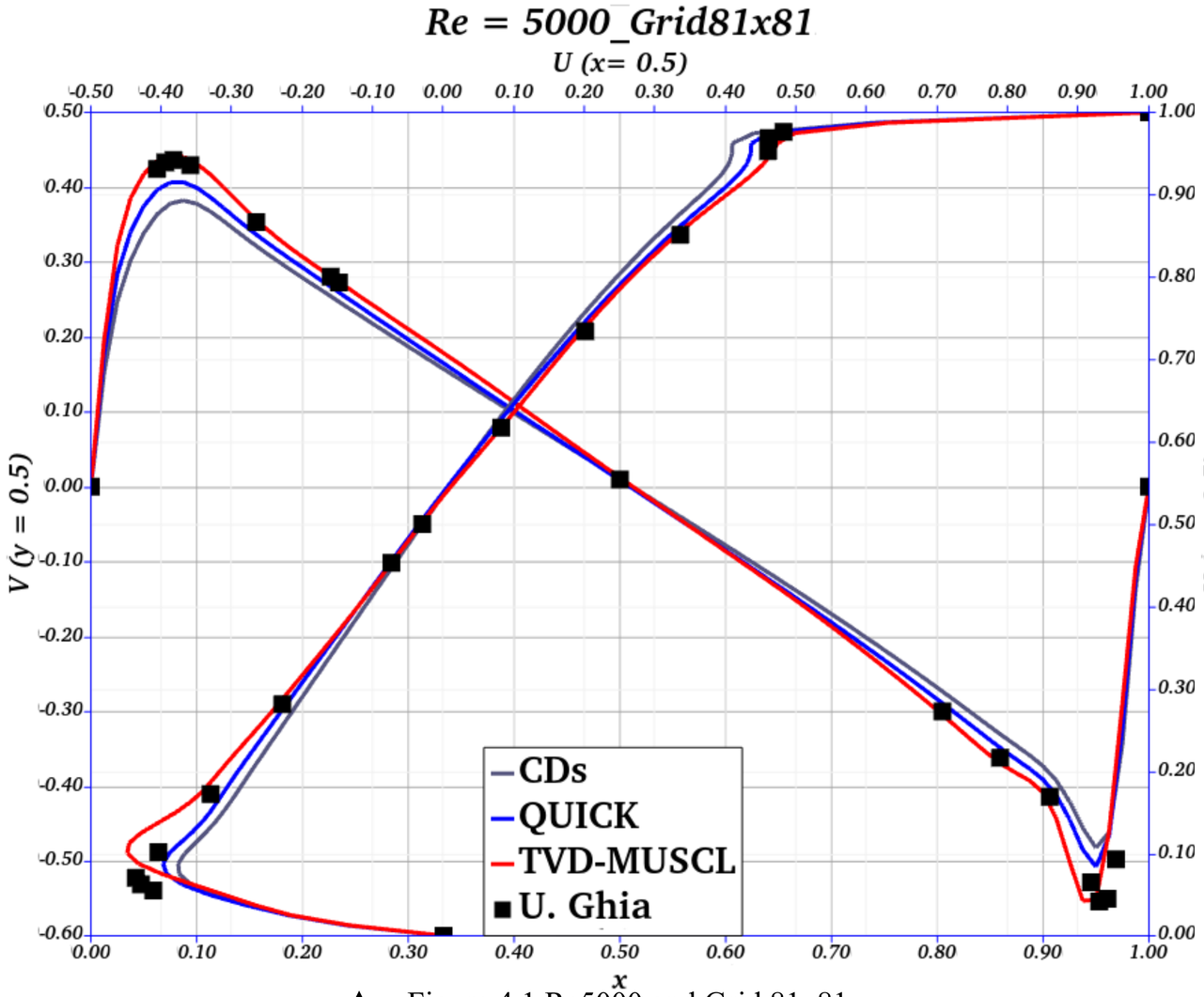
▲ Table 4.1 Results of the u-Velocity

129-grid pt. no.	x	Re = 100	400	1000	3200	5000	7500	10,000
129	1.0000	0.00000	0.00000	0.00000	0.00000	0.00000	0.00000	0.00000
125	0.9688	−0.05906	−0.12146	−0.21388	−0.39017	−0.49774	−0.53858	−0.54302
124	0.9609	−0.07391	−0.15663	−0.27669	−0.47425	−0.55069	−0.55216	−0.52987
123	0.9531	−0.08864	−0.19254	−0.33714	−0.52357	−0.55408	−0.52347	−0.49099
122	0.9453	−0.10313	−0.22847	−0.39188	−0.54053	−0.52876	−0.48590	−0.45863
117	0.9063	−0.16914	−0.23827	−0.51550	−0.44307	−0.41442	−0.41050	−0.41496
111	0.8594	−0.22445	−0.44993	−0.42665	−0.37401	−0.36214	−0.36213	−0.36737
104	0.8047	−0.24533	−0.38598	−0.31966	−0.31184	−0.30018	−0.30448	−0.30719
65	0.5000	0.05454	0.05186	0.02526	0.00999	0.00945	0.00824	0.00831
31	0.2344	0.17527	0.30174	0.32235	0.28188	0.27280	0.27348	0.27224
30	0.2266	0.17507	0.30203	0.33075	0.29030	0.28066	0.28117	0.28003
21	0.1563	0.16077	0.28124	0.37095	0.37119	0.35368	0.35060	0.35070
13	0.0938	0.12317	0.22965	0.32627	0.42768	0.42951	0.41824	0.41487
11	0.0781	0.10890	0.20920	0.30353	0.41906	0.43648	0.43564	0.43124
10	0.0703	0.10091	0.19713	0.29012	0.40917	0.43329	0.44030	0.43733
9	0.0625	0.09233	0.18360	0.27485	0.39560	0.42447	0.43979	0.43983
1	0.0000	0.00000	0.00000	0.00000	0.00000	0.00000	0.00000	0.00000

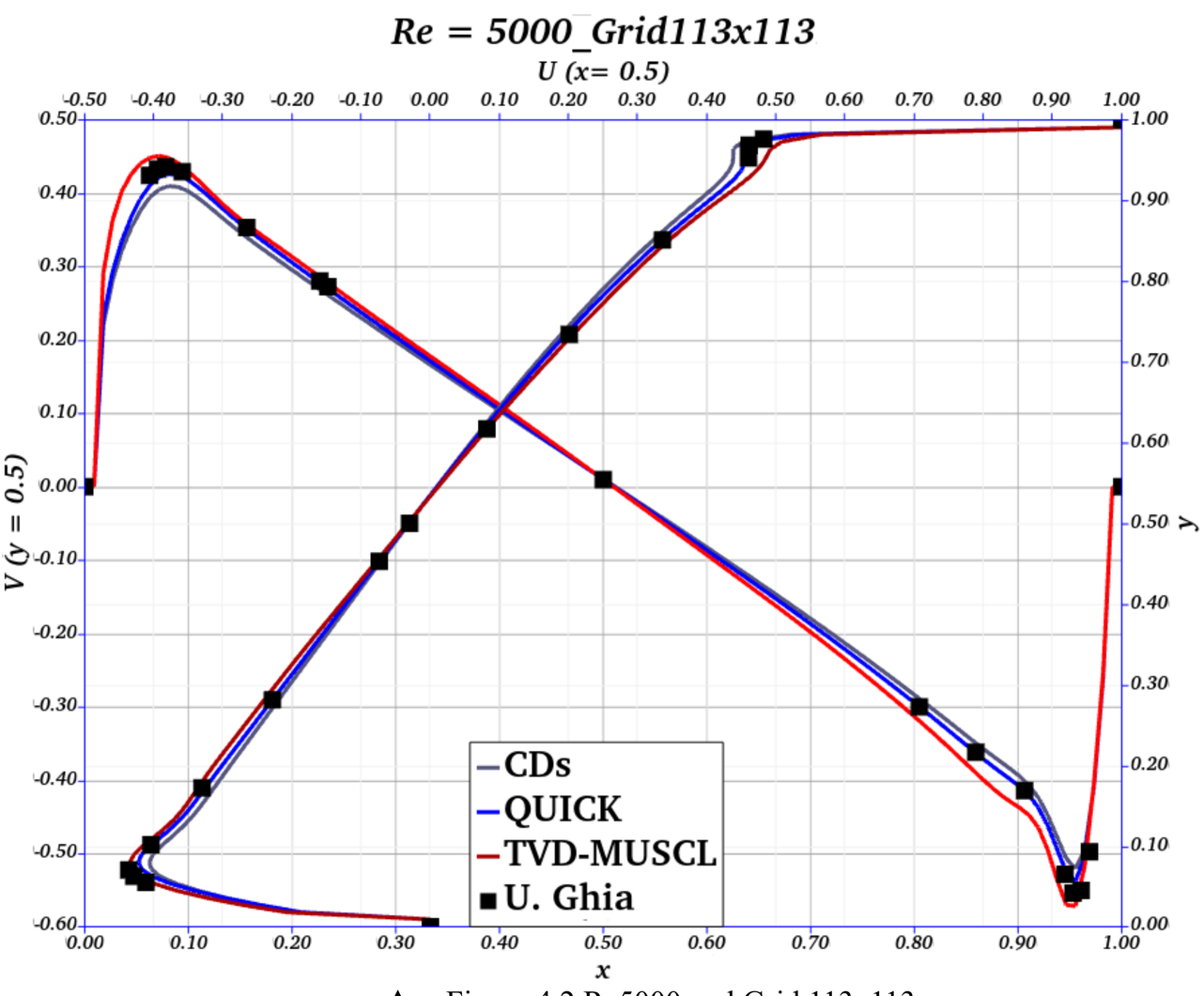
▲ Table 4.2 Results of the v-Velocity

◆ High-Re solutions for incompressible flow using the Navier-Stokes equations and a multigrid method(Ghia et al., 1982)

4. Results and Discussion-Centerline Velocity : Different Scheme



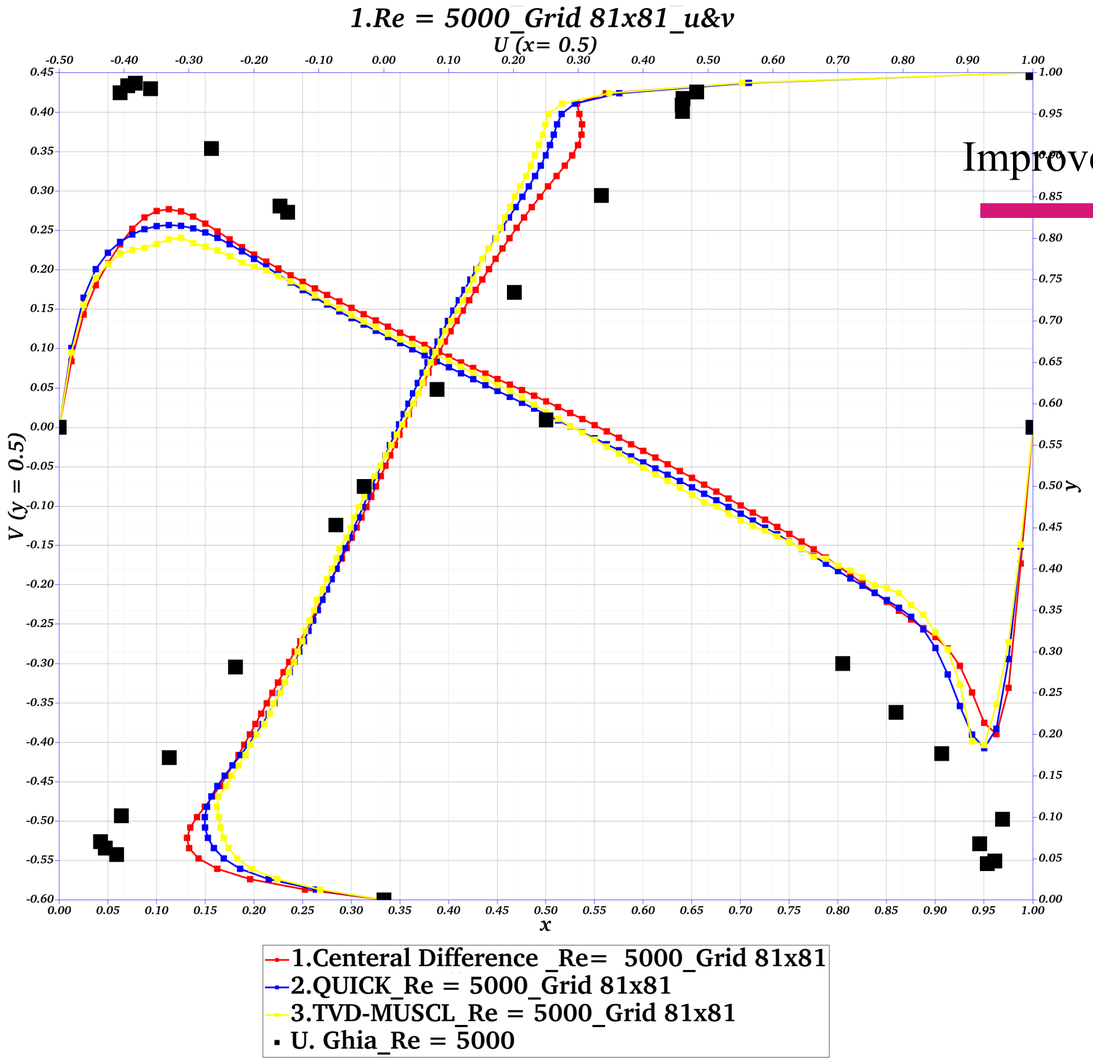
▲ Figure 4.1 Re5000 and Grid 81x81



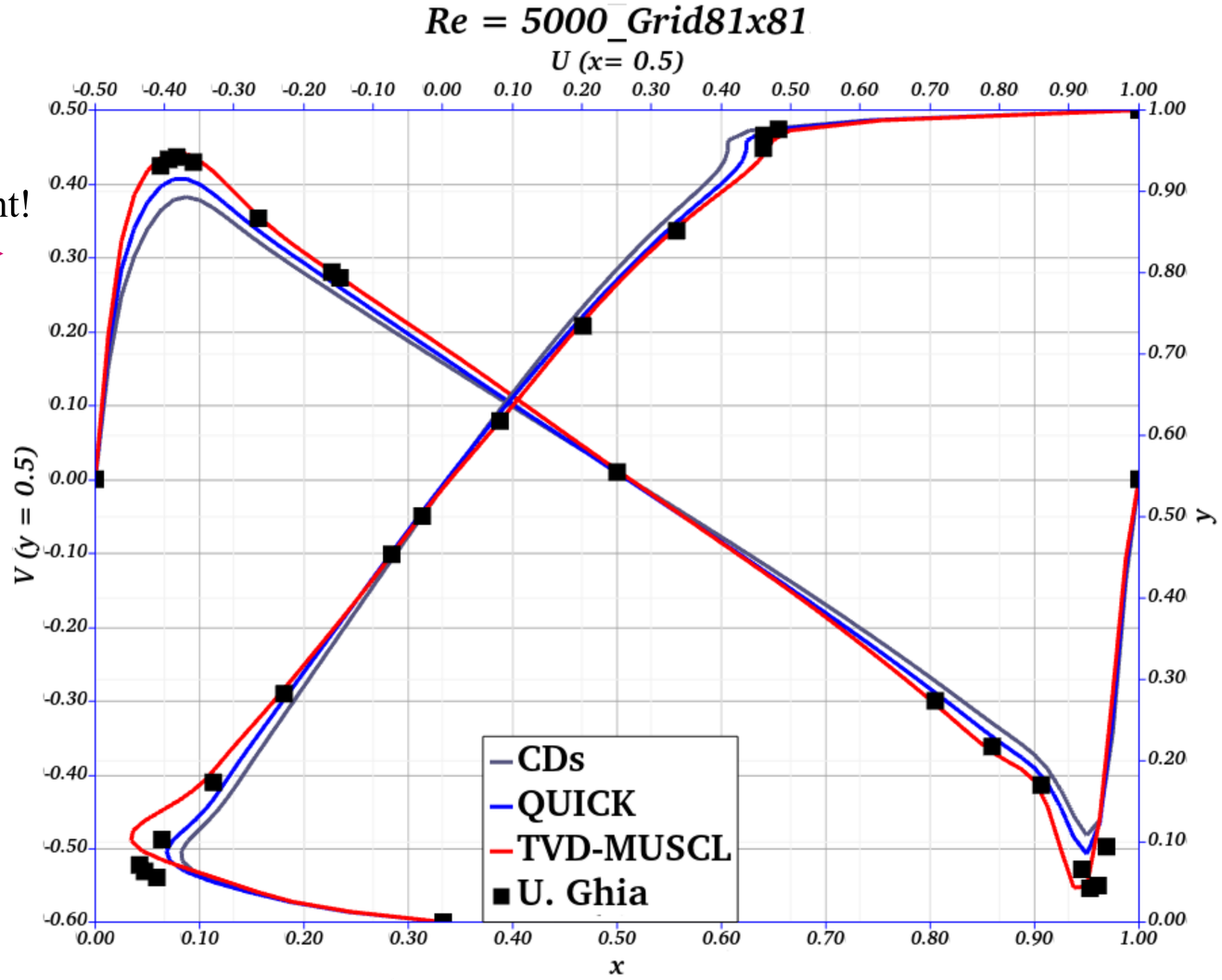
▲ Figure 4.2 Re5000 and Grid 113x113

4. Results and Discussion-Centerline Velocity : success and failure results

- Grid 81x81 : The method I used is more and more fast and efficient !

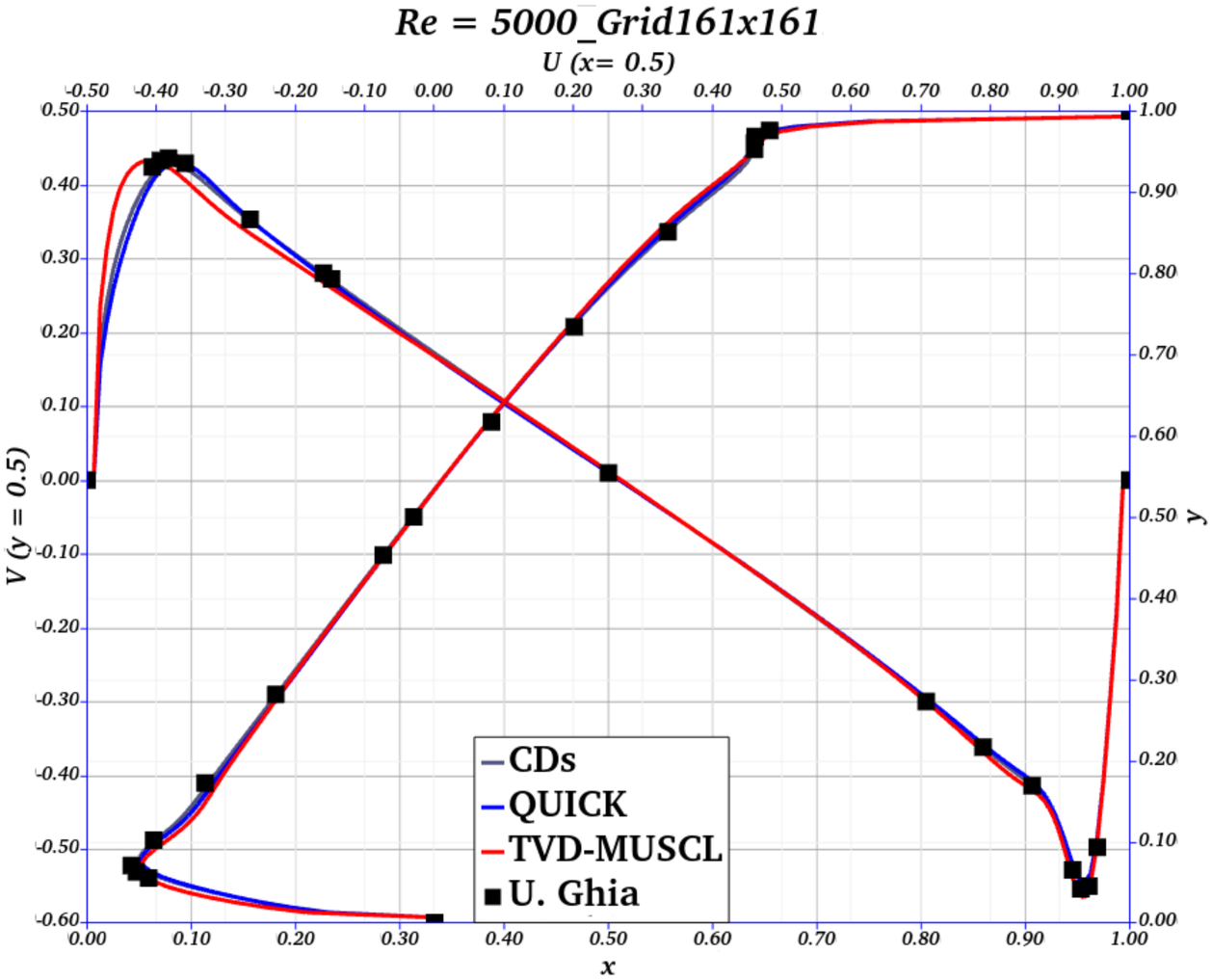


Improvement!

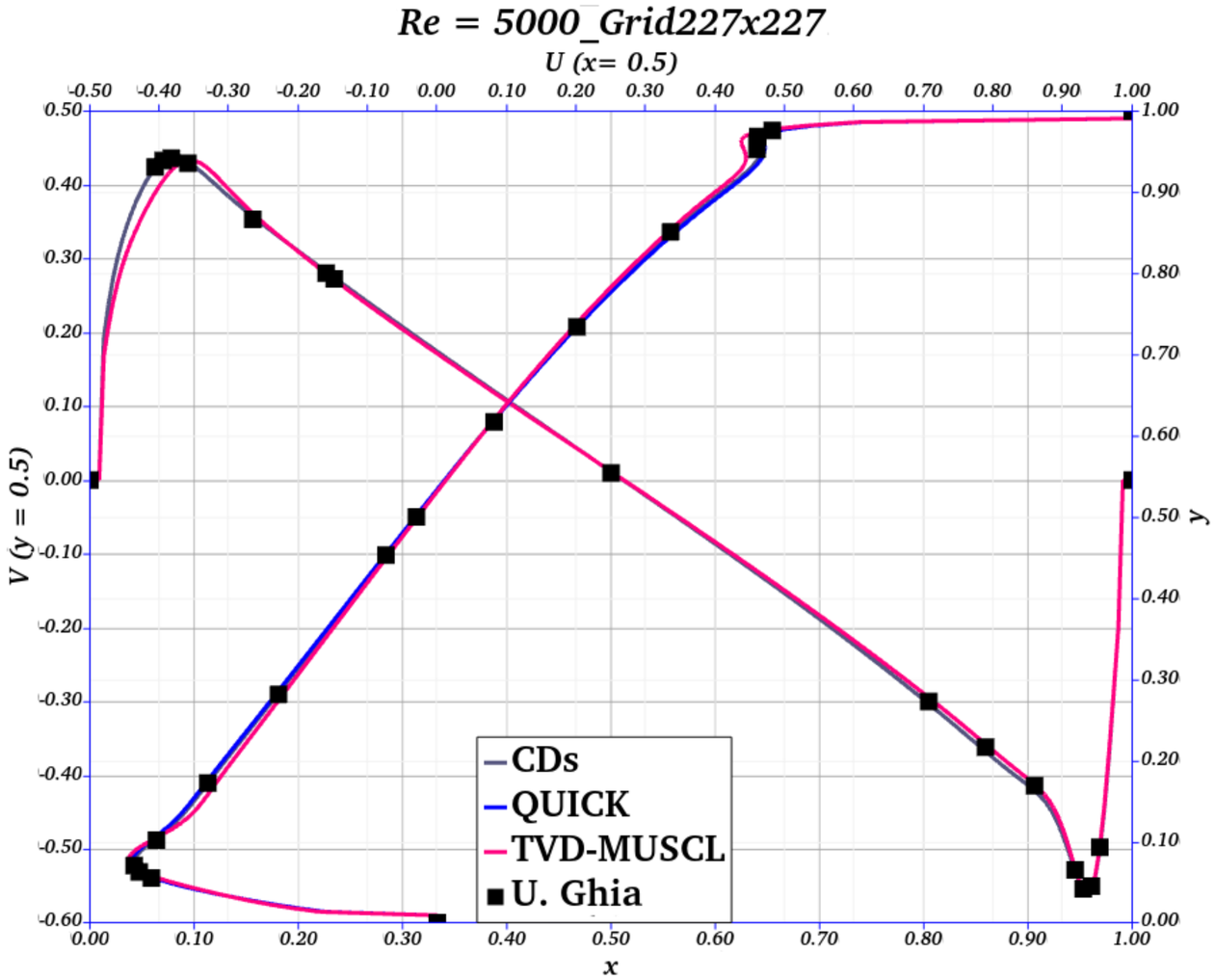


◀ Figure 4.3 Re5000 and Grid 81x81 success case and failure case

4. Results and Discussion-Centerline Velocity : Different Scheme



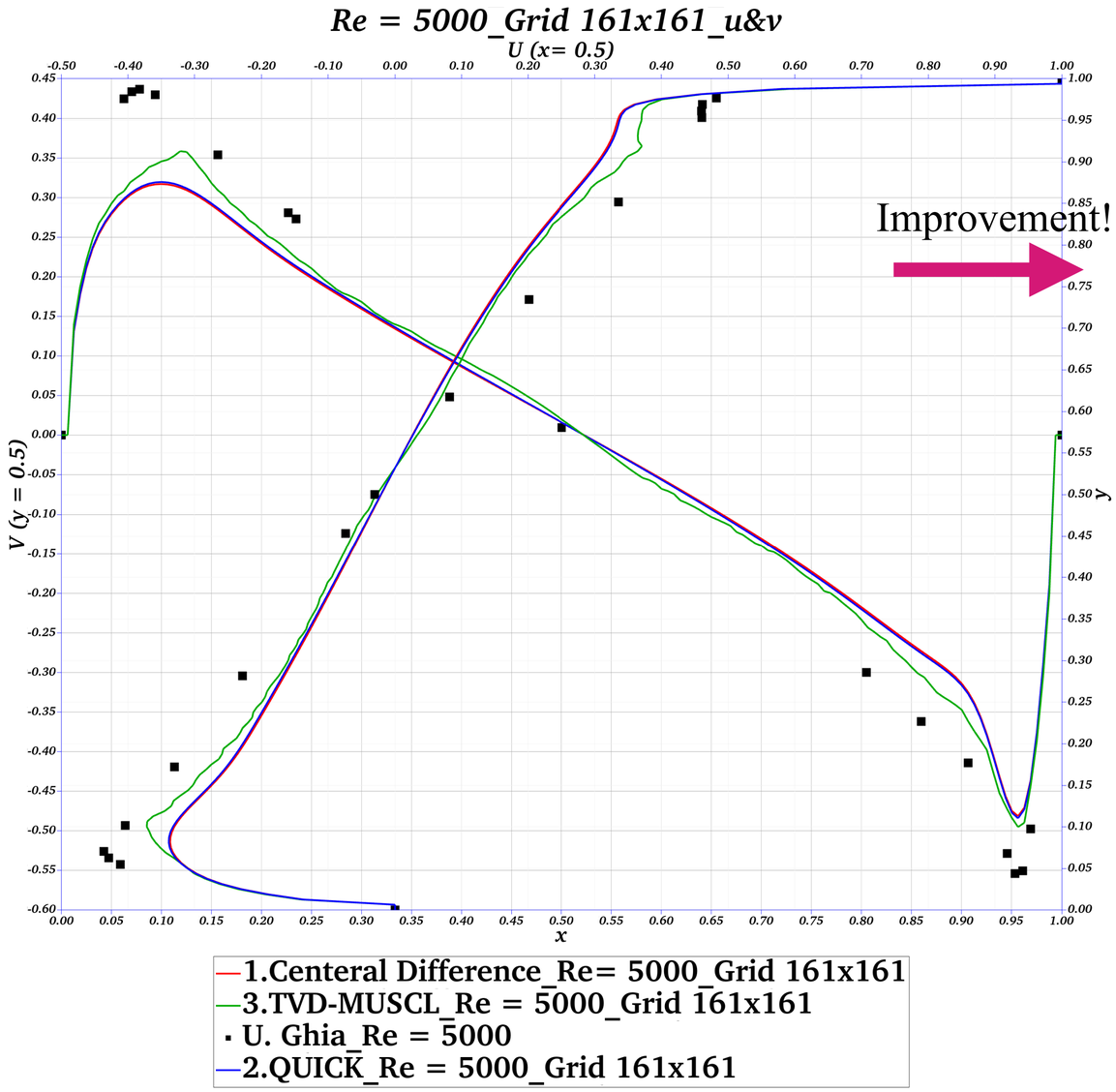
▲ Figure 4.4 Re5000 and Grid 161x161



▲ Figure 4.5 Re5000 and Grid 227x227

4. Results and Discussion-Centerline Velocity : success and failure results

- Grid 161x161 : The first problem about oscillation has been successfully solved.



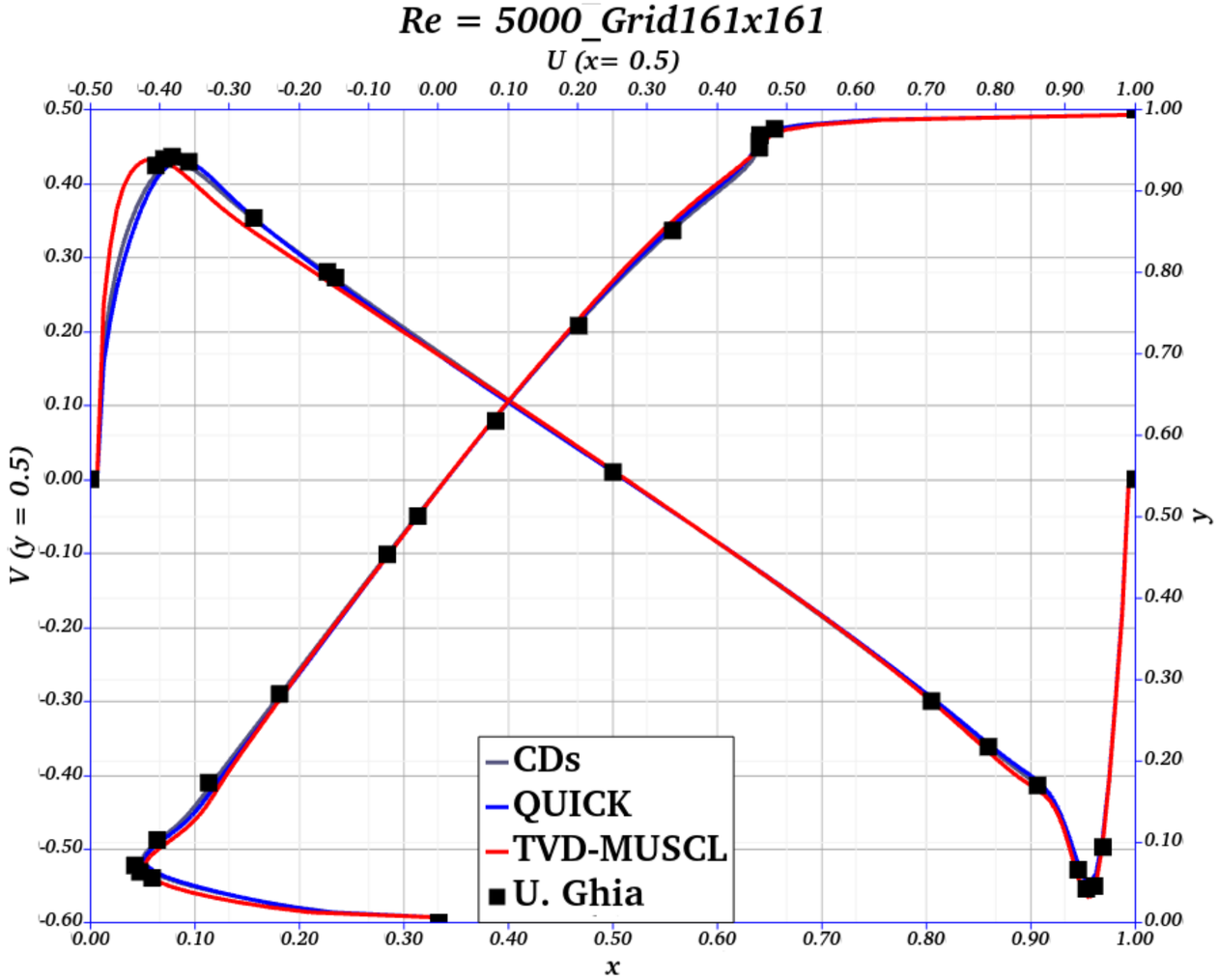
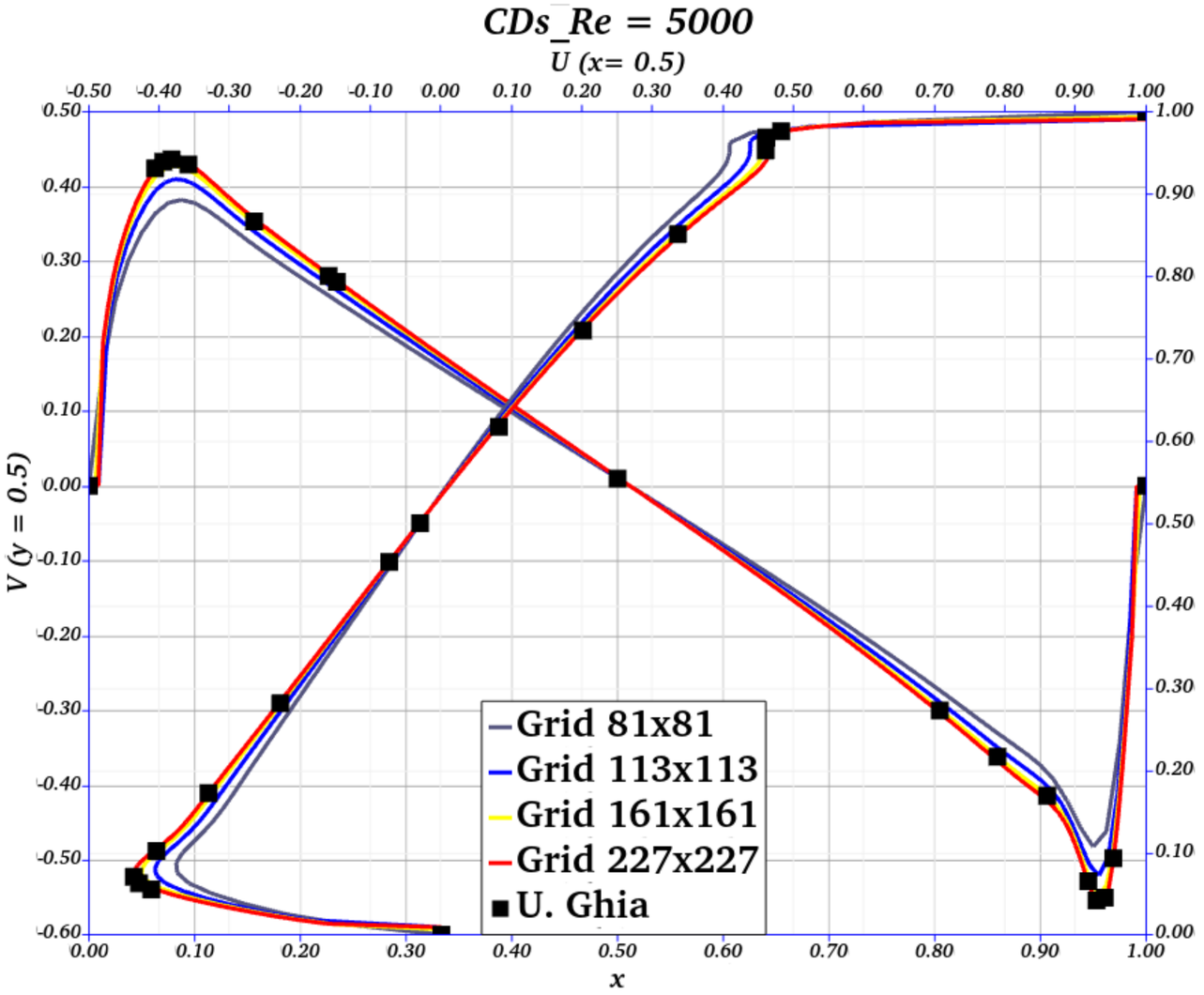
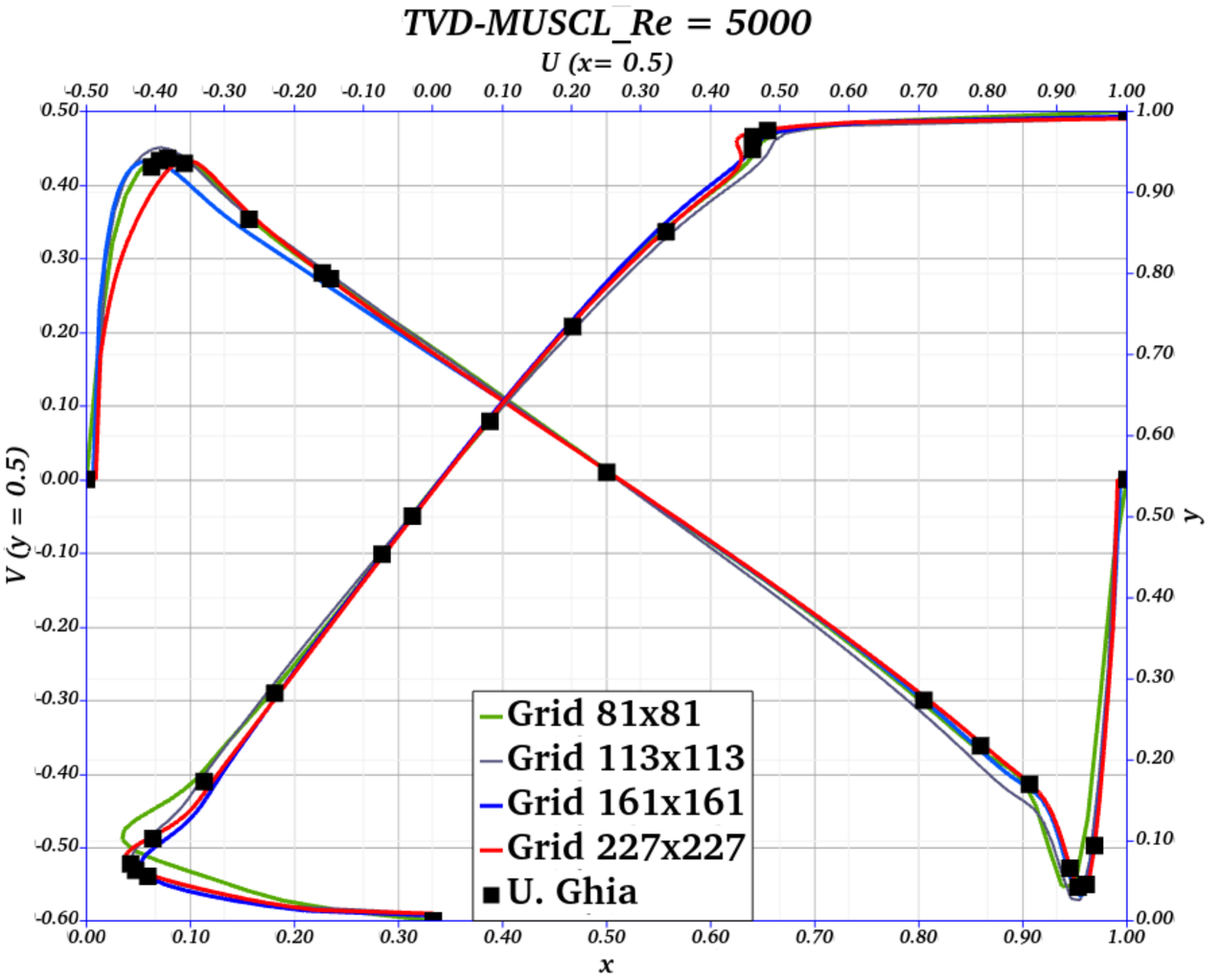


Figure 4.6 Re5000 and Grid 161x161 success case and failure case

4. Results and Discussion-Centerline Velocity : Different Grid Size



▲ Figure 4.7 Central Difference at different grid size



▲ Figure 4.8 TVD-MUSCL at different grid size

4. Results and Discussion-Centerline Velocity : Different Grid Size

- Problems Solved - Summary

1.**Oscillation** : ✅

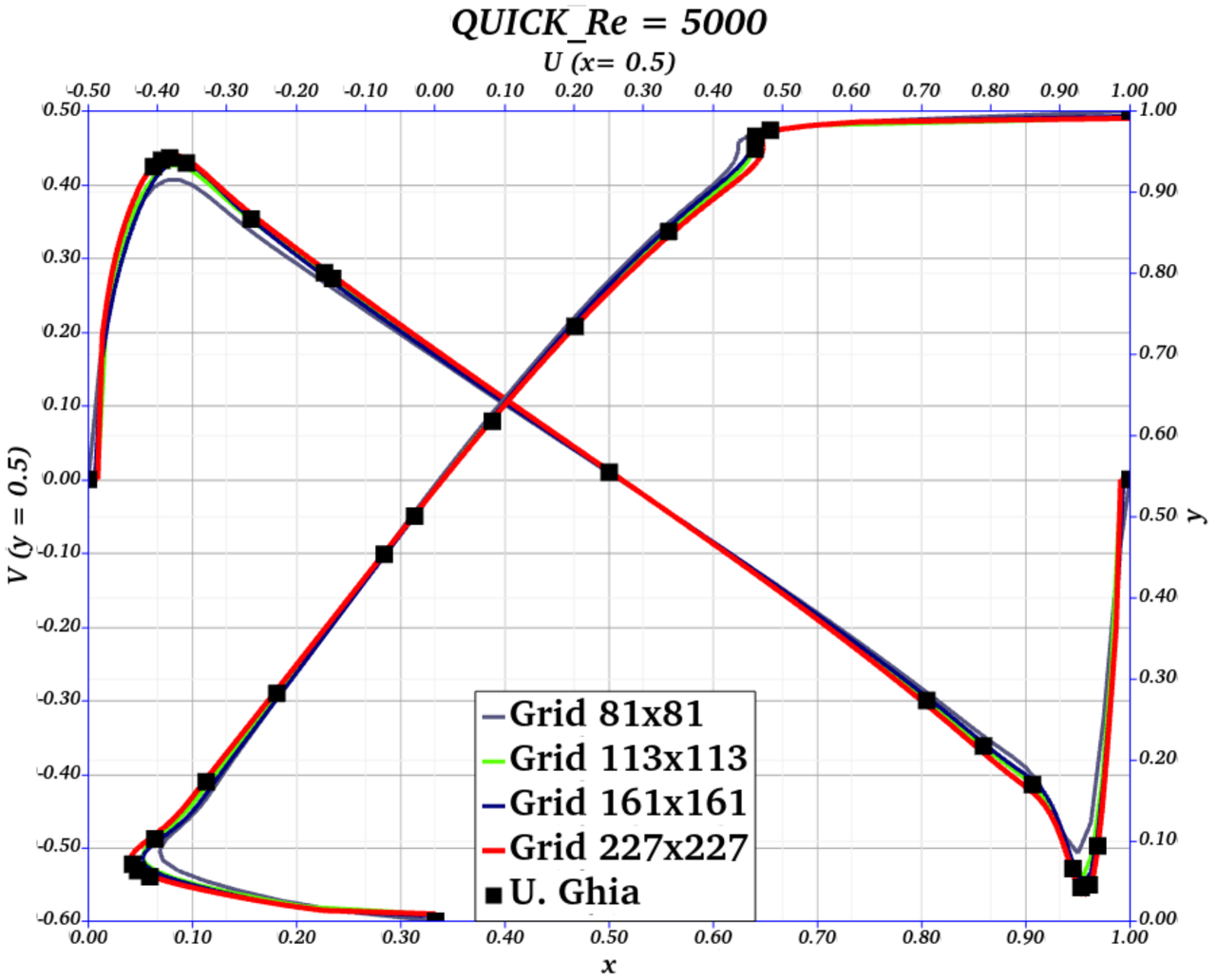
Although the TVD-MUSCL scheme in Figure 4.8 still shows some error compared with the benchmark, it no longer exhibits non-physical phenomena.

2.**BottomLeft x-Velocity** : ✅

All of my x-velocity results fail to achieve the target value of -4.3 at the bottom-left corner shown in the right figure. This is the primary difficulty I aim to address. And My hole profile can perfectly fit with the benchmark.

3.Computational Time Improvent :

Originally, my first code took at least 3 days to obtain a converged solution, which was very inefficient. The limitations of computational resources are issues we must consider. The Results ? Answer in Q & A.



▲ Figure 4.9 QUICK at different grid size

4. Results and Discussion-Accuracy Comparison : CDs, QUICK, TVD

■ Three Problems Identified:

(1) Grid 227×227: CDS error < QUICK error vs. benchmark

(2) Accuracy analysis: CDS slope > QUICK (possibly due to limited benchmark points)

(3) Relaxation sensitivity: Inappropriate settings → wrong solutions or poor benchmark agreement

(4) Experience : The relaxation factor for pressure must be larger than that for velocity; otherwise, the solution will diverge. And The larger the difference between them, the smaller the error at the final iteration.

